

ReconForge Architecture

Version 1.0 — Last updated: 2026-03-21

System Overview

ReconForge is a modular penetration testing reconnaissance framework built in Python. It automates the reconnaissance phase of engagements through five specialized modules and a cross-module workflow orchestrator, all sharing a common core services layer.

Validated Architecture

All five modules follow the same layered pipeline:

```
tools/ → parsers/ → phases/ → module.py → core/
```

Two modules extend this base pattern:

- **Surface** adds an `intelligence/` layer between parsers and phases
- **AD** extends to: `tools/ → parsers/ → collectors/ → analyzers/ → attack_paths/ → phases/ → reporting/ → module.py → core/`

Architecture Overview Diagram

```

graph TB
  CLI["reconforge.py<br/>(CLI Entry Point)"]
  WO["WorkflowOrchestrator<br/>core/workflow_orchestrator.py"]

  CLI --> WO

  subgraph Modules
    direction LR
    NET["Network<br/>network_module.py"]
    WEB["Web<br/>web_module.py"]
    API["API<br/>api_module.py"]
    SRF["Surface<br/>surface_module.py"]
    AD["AD<br/>ad_module.py"]
  end

  WO --> NET
  WO --> WEB
  WO --> API
  WO --> SRF
  WO --> AD

  subgraph net_layers ["Network Pipeline"]
    direction TB
    NT["tools/<br/>nmap, enum4linux,<br/>smbclient, ldapsearch, hydra"]
    NP["parsers/<br/>nmap, enum4linux,<br/>smb, ldap"]
    NPH["phases/<br/>host_discovery, port_scanning,<br/>service_enumeration, authentication_checks"]
    NT --> NP --> NPH
  end

  subgraph web_layers ["Web Pipeline"]
    direction TB
    WT["tools/<br/>gobuster, ffuf, nikto, nuclei,<br/>whatweb, wafw00f, wpscan, sqlmap, curl"]
    WP["parsers/<br/>gobuster, ffuf, nikto, nuclei,<br/>whatweb, wafw00f, wpscan"]
    WPH["phases/<br/>surface_discovery, content_enumeration,<br/>vulnerability_scanning, exploit_candidates"]
    WT --> WP --> WPH
  end

  subgraph api_layers ["API Pipeline"]
    direction TB
    AT["tools/<br/>httpx, arjun,<br/>ffuf_api, nuclei_api"]
    AP["parsers/<br/>openapi, arjun,<br/>ffuf, nuclei"]
    APH["phases/<br/>discovery, authentication,<br/>fuzzing, authorization"]
    AT --> AP --> APH
  end

  subgraph srf_layers ["Surface Pipeline"]
    direction TB
    ST["tools/<br/>nmap_stealth,<br/>service_detector"]
    SP["parsers/<br/>surface_parser"]
    SI["intelligence/<br/>correlation_engine, deduplicator,<br/>attack_prioritizer, confidence_scorer,<br/>service_normalizer, service_intelligence"]
    SPH["phases/<br/>port_discovery, service_fingerprint,<br/>vector_correlation, prioritization"]
    ST --> SP --> SI --> SPH
  end

  subgraph ad_layers ["AD Pipeline"]
    direction TB
    ADT["tools/<br/>bloodhound, enum4linux_ng, impacket,<br/>ldapsearch, netexec,

```

```

nmap, smbclient"]
    ADP["parsers/<br/>bloodhound, delegation, enum4linux_ng,<br/>impacket, ldap,
netexec, nmap, smb"]
    ADC["collectors/<br/>bloodhound, delegation, dns,<br/>kerberos, ldap, smb"]

ADA["analyzers/<br/>misconfiguration, permission,<br/>privilege, relationship, trust"]
    ADAP["attack_paths/<br/>acl, asrep, delegation,<br/>gpo, kerberoast, priv-
ilege_escalation"]
    ADPH["phases/<br/>passive_recon, identity_enumeration,<br/>configura-
tion_enumeration,<br/>delegation_discovery, bloodhound_collection"]
    ADR["reporting/<br/>ad_summary, attack_path, attack_surface,<br/>
>high_value_targets, remediation"]
    ADT --> ADP --> ADC --> ADA --> ADAP --> ADPH --> ADR
end

NET --- net_layers
WEB --- web_layers
API --- api_layers
SRF --- srf_layers
AD --- ad_layers

subgraph Core ["core/ - 18 Shared Services"]
    R["runner.py"]
    CL["config_loader.py"]
    TC["tool_config.py"]
    PL["profile_loader.py"]
    FM["findings_manager.py"]
    LM["loot_manager.py"]
    NM["notes_manager.py"]
    OM["output_manager.py"]
    AW["attack_workflow.py"]
    CV["credential_vault.py"]
    EM["engagement.py"]
    OC["opsec_checks.py"]
    TP["target_parser.py"]
    LG["logger.py"]
    DM["detection_map.py"]
    EX["exceptions.py"]
    VL["validators.py"]
    UT["utils.py"]
end

net_layers --> Core
web_layers --> Core
api_layers --> Core
srf_layers --> Core
ad_layers --> Core

```

Directory Structure

```

reconforge/
├── reconforge.py          # CLI entry-point (argparse)
├── config/
│   ├── tools.yaml        # Tool binary paths, timeouts, scan profiles
│   └── profiles.yaml     # OPSEC-aware scan profiles
├── core/
│   ├── runner.py         # Shared services layer
│   └── shell=True)
│       ├── config_loader.py    # YAML config loading with caching
│       ├── tool_config.py      # Typed accessor for tool YAML entries
│       ├── profile_loader.py   # OPSEC profile resolution
│       ├── findings_manager.py # 5-level confidence model, severity clamping
│       └── loot_manager.py     # Credential/hash/token tracking, optional Fernet
├── encryption
│   ├── notes_manager.py      # Timestamped session notes
│   ├── output_manager.py     # Structured output directories, engagement reports
│   ├── attack_workflow.py    # Kill-chain tracking, hypothesis management
│   ├── credential_vault.py   # Centralized credential store with deduplication
│   └── engagement.py         # Engagement lifecycle (planning → active → com-
pleted)
│       ├── workflow_orchestrator.py # Cross-module chaining with conditional branching
│       ├── validators.py       # IP, CIDR, hostname, URL, port validation
│       ├── opsec_checks.py    # Technique-level OPSEC gating
│       ├── detection_map.py   # Noise-level mapping for all techniques
│       ├── exceptions.py      # Structured exception hierarchy
│       ├── logger.py          # Color-coded logging with credential sanitization
│       ├── target_parser.py   # Target string parsing (IP, CIDR, hostname)
│       └── utils.py           # Utility helpers
├── modules/
│   ├── network/
│   │   ├── base.py          # Network reconnaissance
│   │   └── network_module.py # NetworkPhaseBase (ABC)
│   │       ├── tools/
│   │       │   ├── base.py          # Module orchestrator
│   │       │   └── parsers/
│   │       │       ├── nmap/
│   │       │       │   ├── base.py          # nmap, enum4linux, smbclient, ldapsearch, hydra
│   │       │       │   └── parsers/
│   │       │       │       ├── nmap_parser, enum4linux_parser, smb_parser,
│   │       │       │       └── ldap_parser
│   │       │           ├── phases/
│   │       │           │   ├── base.py          # host_discovery, port_scanning, ser-
│   │       │           │   └── vice_enumeration, authentication_checks
│   │       │           └── web/
│   │       │               ├── base.py          # Web application reconnaissance
│   │       │               └── web_module.py    # WebPhaseBase (ABC)
│   │       │                   ├── tools/
│   │       │                   │   ├── base.py          # Module orchestrator
│   │       │                   │   └── parsers/
│   │       │                   │       ├── gobuster, ffuf, nikto, nuclei, whatweb, wafw00f, wpscan, sqlmap, curl_tool
│   │       │                   │       └── parsers/
│   │       │                   │           ├── gobuster, ffuf, nikto, nuclei, whatweb, wafw00f, wpscan
│   │       │                   │           └── phases/
│   │       │                   │               # surface_discovery, content_enumeration, vulner-
│   │       │                   │               ability_scanning, exploit_candidates
│   │       │                   └── api/
│   │       │                       # API security assessment
│   │       │                       ├── base.py          # APIPhaseBase (ABC)
│   │       │                       └── api_module.py    # Module orchestrator
│   │       │                           ├── tools/
│   │       │                           │   ├── ffuf_api, httpx_tool, arjun_tool, nuclei_api
│   │       │                           │   └── parsers/
│   │       │                           │       ├── openapi_parser, arjun_parser, ffuf_parser, nuc-
│   │       │                           │       └── lei_parser
│   │       │                           └── phases/
│   │       │                           │   # discovery, authentication, fuzzing, authoriza-
│   │       │                           │   tion
│   │       │                           └── surface/
│   │       │                               # Attack surface mapping
│   │       │                               ├── base.py          # SurfacePhaseBase (ABC)
│   │       │                               └── surface_module.py # Module orchestrator
│   │       │                                   ├── tools/
│   │       │                                   │   ├── nmap_stealth, service_detector
│   │       │                                   │   └── parsers/
│   │       │                                   │       # surface_parser
│   │       │                                   │       ├── intelligence/
│   │       │                                   │       │   ├── correlation_engine, confidence_scorer, deduplic-
│   │       │                                   │       │   └── ator, service_normalizer, service_intelligence, attack_prioritizer
│   │       │                                   │       └── phases/
│   │       │                                   │           # port_discovery, service_fingerprint, vec-

```

```

tor_correlation, prioritization
├── ad/                                # Active Directory reconnaissance
│   ├── base.py                       # ADPhaseBase (ABC)
│   ├── ad_module.py                 # Module orchestrator
│   └── tools/
# nmap, enum4linux_ng, impacket, advanced_impacket, ldapsearch, smbclient, bloodhound,
netexec
├── parsers/                          # nmap, enum4linux_ng, impacket, smb, ldap,
bloodhound, delegation, netexec
├── collectors/                      # bloodhound, delegation, dns, kerberos, ldap, smb
├── analyzers/                       # misconfiguration, permission, privilege,
relationship, trust
├── attack_paths/                    # acl, asrep, delegation, gpo, kerberoast, priv-
ilege_escalation
├── phases/                          # passive_recon, identity_enumeration, configura-
tion_enumeration, delegation_discovery, bloodhound_collection
├── reporting/                       # ad_summary, attack_path, attack_surface,
high_value_targets, remediation, report_builders
└── tests/                          # 348 tests (pytest)

```

Core Services

Runner (core/runner.py)

Secure subprocess execution. **All commands are executed as** `list[str]` **via** `subprocess.run` — **never** `shell=True`.

- String commands are accepted for backward compatibility but emit a `DeprecationWarning` and are split via `shlex.split`
- `validate_arg()` rejects shell metacharacters (; & | \ \$ () { } `) before they reach subprocess
- Returns `RunResult` dataclass with stdout, stderr, returncode, duration
- `run_or_raise()` variant throws typed exceptions: `ToolNotFoundError`, `TimeoutError`, `ExecutionError`
- Supports dry-run mode, custom timeouts, stdin piping, environment overrides
- Maintains a command log exportable via `get_command_log()` / `save_command_log()`

ConfigLoader (core/config_loader.py)

Single authoritative source for all YAML configuration. Loads `tools.yaml` and `profiles.yaml` with file-level caching. No fallback namespaces — the config file is the source of truth.

ToolConfig (core/tool_config.py)

Typed accessor for a tool's YAML configuration. Provides:

- `binary`, `alt_binary`, `required`, `default_timeout`, `description`, `detection`, `opt_in_only`
- `mode_timeout(mode, default)` — timeout resolution: mode-specific → tool default → caller default
- `mode_args(mode)`, `mode_detection(mode)`, `mode_value(mode, key)`
- `safety(key)` — for safety blocks (e.g., hydra max_tasks)
- `collection(method, key)` — for collection methods (e.g., bloodhound)
- `effective_timeout(mode, caller_default)` — unified timeout resolution
- Full backward compatibility: `ToolConfig(None, "tool")` returns caller defaults

ProfileLoader (core/profile_loader.py)

Resolves OPSEC profiles with module-specific overrides:

1. Module-specific variant (`stealth_ad` , `normal_web` , etc.)
2. Exact profile name
3. Base mode via canonical mapping
4. Empty dict (all defaults)

Provides: `timing` , `allowed_noise` , `nmap_timing` , `scan_delay` , `is_technique_enabled()` , `enabled_phases()` .

FindingsManager (core/findings_manager.py)

5-level confidence model with strict severity clamping. See [FINDINGS.md](#) (FINDINGS.md) for details.

LootManager (core/loot_manager.py)

Tracks credentials, hashes, tokens, shares, users, and services. Supports:

- Automatic deduplication
- Optional Fernet encryption (`--encrypt-loot`), key stored at `~/.reconforge/loot.key`
- Typed add methods: `add_credential()` , `add_hash()` , `add_user()` , `add_share()` , `add_service()`

CredentialVault (core/credential_vault.py)

Centralized credential store shared across modules. Supports:

- Types: `password` , `hash_ntlm` , `hash_ntlmv2` , `hash_other` , `token_jwt` , `token_bearer` , `api_key` , `ssh_key` , `username` , `cookie` , `certificate`
- Automatic deduplication via fingerprinting
- `ingest_from_loot(loot_manager)` — auto-import from LootManager
- `contribute_to_loot(loot_manager)` — push back to LootManager
- `get_for_service(service)` — service-specific credential retrieval
- Optional Fernet encryption, export/import (JSON, username lists, password lists, hash files)

EngagementManager (core/engagement.py)

Full lifecycle tracking: `planning` → `active` → `paused` → `completed` → `cancelled`

- Pause/resume via JSON serialization (`save()` / `load()`)
- Timeline recording, module result aggregation
- Findings and loot summary aggregation across modules

WorkflowOrchestrator (core/workflow_orchestrator.py)

Cross-module chaining with:

- `WorkflowContext` — shared data bus (live hosts, open ports, services, domains, URLs)
- Conditional branching: AD runs if LDAP/Kerberos/SMB detected; Web/API if HTTP detected
- Automatic credential vault integration — credentials flow between modules
- `full_recon()` — pre-configured pipeline: `surface` → `network` → `ad` → `web` → `api`
- `targeted()` — run specific modules without conditions

AttackWorkflow (core/attack_workflow.py)

Kill-chain tracking with:

- Workflow steps with hypotheses, justifications, alternatives
- Attack path recording (name, description, steps, risk, prerequisites)
- Next-command suggestions (prioritized)
- Rabbit-hole tracking

NotesManager (core/notes_manager.py)

Timestamped session notes with categories: `phase`, `finding`, `command`, `general`. Markdown export with timeline icons.

OutputManager (core/output_manager.py)

Consistent directory structure:

```
outputs/<target>/<module>/
├── raw/           # Raw tool output
├── parsed/        # Parsed results
├── findings.json  # JSON findings
├── findings.md    # Markdown findings
├── loot.json      # Discovered loot
├── session.md     # Session notes
├── commands.log   # Command log
├── attack_paths.md # Attack path documentation
└── quick_report.md # Quick summary report
```

Engagement-level report: `outputs/<target>/engagement_report.md`

Validators (core/validators.py)

Input validation: `validate_ip()`, `validate_cidr()`, `validate_hostname()`, `validate_target()`, `validate_port()`, `validate_port_range()`, `validate_url()`, `validate_domain()`. All raise typed `ValidationError` subclasses.

Exception Hierarchy (core/exceptions.py)

```
ReconForgeError
├── ConfigError
│   └── ProfileNotFoundError
├── ValidationError
│   ├── TargetValidationError
│   └── PortValidationError
├── ExecutionError
│   ├── ToolNotFoundError
│   └── TimeoutError
├── ModuleError
│   └── PhaseError
├── WorkflowError
│   └── WorkflowAbortedError
├── CredentialVaultError
├── EngagementError
│   └── EngagementNotFoundError
```

Logger (core/logger.py)

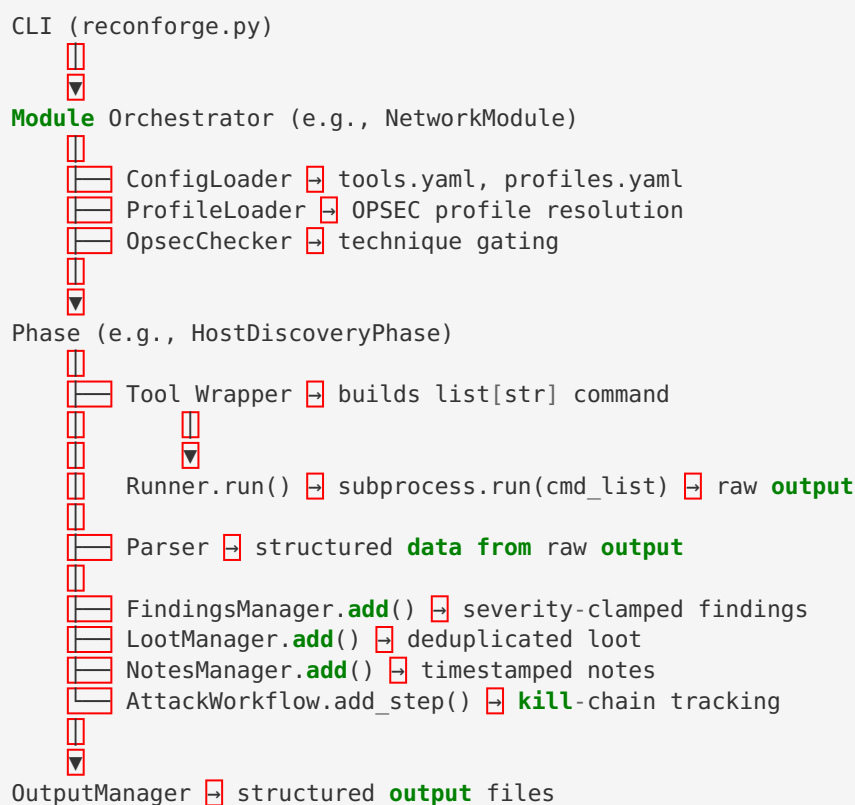
Color-coded console output with file logging. Features:

- Credential sanitization in all log output (passwords, hashes, tokens, API keys redacted)
- Context logging via `with_context()` for module/phase prefixes
- Structured methods: `command()`, `finding()`, `loot()`, `credential()`, `phase_start()`, `phase_end()`

OPSEC System (core/opsec_checks.py , core/detection_map.py)

Every tool/technique has a noise level (`low`, `medium`, `high`, `very_high`). The `OpsecChecker` gates execution based on the active profile's `allowed_noise_levels`. Blocked techniques log a warning with the reason.

Data Flow



Phase Base Classes

All modules use a consistent phase base class with an identical constructor signature (11 parameters):

Parameter	Type	Description
logger	ReconLogger	Logging instance
runner	Runner	Subprocess executor
config	ConfigLoader	YAML config access
output_dir	Path	Parsed output directory
findings	FindingsManager	Finding tracker
loot	LootManager	Loot tracker
workflow	AttackWorkflow	Kill-chain tracker
notes	NotesManager	Session notes
opsec	OpsecChecker	OPSEC gating
opsec_mode	str	Current mode string
profile	ProfileLoader	Resolved OPSEC profile

Each phase base defines class attributes: `PHASE_NUMBER` , `PHASE_NAME` , `PHASE_DESCRIPTION` and requires subclasses to implement a `run()` method.

Command Execution Safety

- **Zero instances** of `shell=True` in production code
- All tool wrappers build commands as `list[str]`
- `Runner.run()` accepts both `str` and `list[str]` , splitting strings via `shlex.split`
- `validate_arg()` rejects shell metacharacters before they reach `subprocess`
- Logger sanitizes credentials from all log output

Testing

- **348 tests, all passing** (pytest, ~3.1s)
- Coverage spans: `config_loader`, `credential_vault`, `engagement`, `logger`, `loot_manager`, `profile_loader`, `runner`, `target_parser`, `validators`, `workflow_orchestrator`, `tool_config`, API module, parsers (nmap, arjun, ffuf, nuclei_api), JWT analysis, OpenAPI parser, authorization/fuzzing, surface intelligence, profile activation

Architecture validated: 2026-03-21 — ReconForge v1.0 Stabilization Audit (348/348 tests passing)